VERIFORM

Verifiable AI Agents with TEE-Backed Decision Receipts

Proof, not blind trust.

Complete Project Guide

Problem • Existing Solutions • Approach • What We Built • Demo

July 2026 | 49 tests, CI green | github.com/Sakthi-Sundaram-R/Veriform

Contents

1. Executive Summary

Veriform makes AI agents **provable**. Every decision an agent makes is wrapped in a **cryptographic receipt** that binds that exact decision to unaltered code running inside a hardware-secured enclave (a Trusted Execution Environment, or TEE). Anyone — an end user, an auditor, or a smart contract — can verify the receipt in seconds and reject any output that was tampered with, forged, or produced outside the enclave.

The reference application is a **wallet-guardian agent** that approves or denies crypto transfer requests. The decision engine is deliberately simple, because the decision is not the product — **the receipt is**. The same mechanism generalizes to any agent whose outputs carry consequences: financial agents, medical triage, compliance audits, autonomous on-chain actors.

Beyond binding a single decision, Veriform contributes four capabilities that, as an integrated and independently-verifiable package, do not have an off-the-shelf equivalent:

- **Attested inference provenance** — binds which model judged, under which hashed prompt; catches a backdoored judgment prompt even inside a genuine enclave.
- **Attested decision ledger** — makes an agent's entire decision *history* verifiable: complete, ordered, and compliant with a cross-decision policy.
- **Attested multi-judge consensus** — requires a quorum of independent judges so no single rogue or hallucinating model can force a decision.
- **Full DCAP verification** — the complete Intel chain of trust, verified offline and proven against a real hardware quote.

The system is fully implemented, tested (49 automated tests, continuous integration green), and demonstrated end to end — with four distinct attacks each rejected for a different, real cryptographic reason.

2. Problem Statement — the trust gap

AI agents increasingly act on their own: approving transactions, handling private user data, executing on-chain operations without a human in the loop. But every agent runs on infrastructure that someone controls. Whoever operates that server can read the agent's inputs, alter its decisions, swap its code, or extract the private keys it holds — and the end user has no way to detect any of it.

When an agent signs a transaction, there is normally no way to prove that:

- the agent ran the exact code it claims to run, unmodified;
- its decision was genuinely produced inside a secure environment, not forged by the host;
- its private keys never left that environment.

Running agents inside TEEs is no longer theoretical — the hardware primitive is solved. What remains unsolved is everything *between that primitive and a person or contract who needs to trust it*. Three gaps persist:

- **Attestation is unusable by the people who need it.** A remote attestation is a raw cryptographic quote; a user or a smart contract has no simple, fast way to consume it. There is no clean “verify in ten seconds” layer.
- **Implementations are quietly insecure.** Analyses of open-source TEE projects find large fractions bypassing official cryptographic APIs. The hardware guarantee is strong; the software wrapping it often silently breaks it.

- **Trust rests on a single vendor.** Nearly all attested inference depends on one vendor's signing infrastructure. A single PKI compromise breaks the whole guarantee.

The challenge: build the verification layer that makes an attested agent's guarantee **consumable** — correct, fast, and hard to fake — for any third party, without trusting the operator.

3. What Veriform Solves

In one line: **it lets you trust an AI agent's decision without trusting whoever runs it.** Concretely, a person or a smart contract receiving an agent's decision can now confirm, in seconds:

Question the receiver now answers	How
Did unaltered, audited code make this decision?	enclave measurement (MRTD) pinning
Was this exact decision produced inside the enclave?	decision binding in the quote's report_data
Which model judged it, under what instructions?	attested inference provenance
Could one rogue model have forced it?	attested multi-judge consensus
Is the agent's whole decision history complete and policy-compliant?	attested decision ledger
Is the attestation genuinely from Intel silicon?	full DCAP verification

Every one of these is verifiable by an outsider who trusts only mathematics and Intel's public PKI — never the operator.

4. Background: TEEs and Remote Attestation

Trusted Execution Environments

A TEE is a hardware-isolated region of a CPU (Intel SGX/TDX, AMD SEV) in which code and data are encrypted and protected even from the machine's own operating system, hypervisor, and root user. Code inside the enclave can hold secrets — signing keys, API credentials — that the host physically cannot read.

Remote attestation

On demand, the CPU produces a **quote**: a data structure signed by the hardware vendor's key hierarchy containing a *measurement* (a hash of the exact code loaded) and a caller-supplied field called **report_data** (64 bytes). A verifier checks the quote's signature chain against Intel's public key infrastructure and compares the measurement to a known-good build. Because the hardware signs whatever the enclave places in report_data, an application can bind arbitrary claims — such as “I made this exact decision” — to the attestation. That is the trick Veriform is built on.

Phala dstack

Veriform uses Phala Network's **dstack** runtime: an ordinary Docker container is deployed unmodified into an Intel TDX confidential VM; inside, a socket exposes the guest-agent API for requesting quotes and deriving sealed keys. A local simulator provides the same API for development, so the identical container image moves from laptop to real silicon with zero code changes.

5. Existing Solutions and the Open Gap

Veriform is honest about what already exists. Being precise about this is part of the contribution — the value is in an integration that is not otherwise available, not in claiming to invent primitives that already exist.

What already exists

Existing work / product	What it provides
Phala, Marlin, Atoma	Run agents inside TEEs and produce remote attestations. The core primitive — solved.
Intel DCAP / Tiber Trust	Standard quote-generation and verification tooling and PKI.
Automata Network	On-chain DCAP attestation verifier (verify a quote in a smart contract).
Confidential inference (emerging)	TEE-hosted model serving so a provider can attest its own inference. Early / limited.
ZK proofs of attestation (research)	Prove you hold a valid quote without revealing it. Active research (zk-DCAP).
Transparency logs, blockchains	Hash-chained, append-only, ordered logs — but not enclave-bound or policy-aware.

What is NOT solved — where Veriform sits

The primitive works; the **consumable trust layer above it** does not. Specifically, no off-the-shelf solution provides, as one verifiable package:

- a **ten-second, human-and-contract-readable** verdict derived from a real attestation;
- binding the **judgment policy** (the LLM prompt), not just the code, into the attestation — so a backdoored prompt in a genuine enclave is caught;
- a **verifiable decision history** with cross-decision policy invariants (completeness + ordering + “never exceeded the cumulative limit”);
- an **attested quorum of judges** so no single model can force a high-stakes decision.

Each of these composes standard primitives (hashing, ECDSA, X.509, quorum) into a form that, to the best of our research, is not otherwise packaged for autonomous agents. That integration — verifiable to outsiders, honest about its boundaries — is the contribution.

6. Solution Approach — the decision receipt

The core idea: bind each decision to the enclave

Attestation normally proves “this code is running unaltered.” Veriform extends it to prove “**this specific decision** was made by that unaltered code.”

```
canonical      = deterministic JSON of the decision payload (sorted keys)
decision_hash  = SHA-256(canonical)
binding        = SHA-256(decision_hash || lowercase(agent_address))
report_data    = binding, padded to 64 bytes    -> placed inside the quote
signature      = secp256k1 over canonical, by a key derived INSIDE the TEE
```

Because report_data is covered by the hardware signature and commits to both the decision hash and the enclave-held key, an attacker cannot alter the decision, replay a quote for a different decision, or sign with a key from outside the enclave.

The receipt

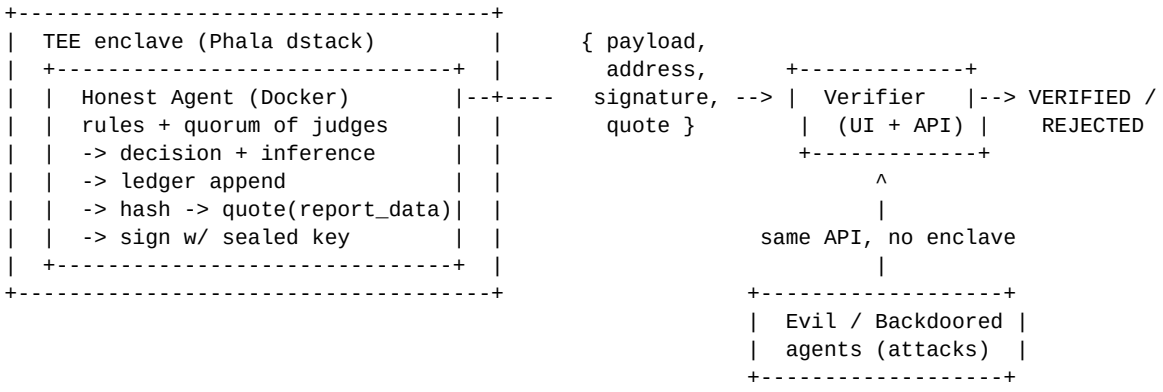
Every agent response carries a signed payload plus proof:

Field	Content
payload	the decision: request, APPROVE/DENY, method, notes, timestamp, and — when present — inference, consensus, and ledger blocks
address	Ethereum-style address of the enclave-derived key
signature	secp256k1 signature over the canonical payload
quote	the TDX attestation quote whose report_data binds the decision

The verifier

The verifier runs *outside* the enclave and trusts nothing but mathematics. It recomputes the binding, extracts report_data from the quote, recovers the signer, validates the Intel chain, and checks the inference / consensus / ledger blocks — rendering one unambiguous verdict (VERIFIED / REJECTED) with a per-check breakdown a non-expert can read in seconds.

7. System Architecture



Four services compose the demo:

Service	Port	Role
agent	8001	honest agent in the enclave/simulator; makes decisions, emits receipts
evil-agent	8002	attacker: identical API, no valid quote or a forged one
backdoored	8003	genuine enclave, valid quote, but a swapped judgment prompt
verifier	3000	verification API + browser UI with the demo toggles

8. What We Built — components

agent/ — the honest agent

- **decision.py** — rules first (address validity, hard limit, trusted auto-approve), then the judgment layer: a pluggable single provider (Claude / Gemini / Ollama / none) or a multi-judge quorum. Every path fails closed — an outage or refusal denies, never approves.
- **ledger.py** — the append-only decision ledger: a hash chain plus a cumulative daily-limit invariant that overrides an approval which would breach the cap.
- **enclave.py** — derives the secp256k1 key inside the TEE, computes the binding, requests the quote, signs the payload.
- **main.py** — assembles the signed payload (decision + inference + consensus + ledger) and attests it.

verifier/ — the verification layer

- **verify.py** — the nine-check pipeline plus verify_sequence() for full decision histories.
- **dcap.py** — complete Intel TDX DCAP verification, offline.
- **main.py + static/index.html** — the API (/ask, /verify-sequence, /verify-dcap) and the single-page demo UI with four agent toggles and a giant VERIFIED/REJECTED verdict.

evil-agent/ + backdoored mode — the attackers

An impostor that approves everything and signs with a non-enclave key (no quote, or a forged one), plus a genuine-enclave agent whose judgment prompt was swapped — together exercising four distinct attack classes.

tests/ + CI

49 automated tests (crypto pipeline, decision rules, inference, ledger, consensus, on-chain signatures, and full DCAP against a real captured hardware quote), run on every push via GitHub Actions.

contracts/ — on-chain anchor

VeriformRegistry.sol and DemoConsumer.sol let a smart contract verify an enclave-signed decision on-chain; the signature scheme is proven ecrecover-compatible.

9. The Verification Pipeline (9 checks)

Checks run in order; any hard failure yields REJECTED. There is no “if attacker” logic — rejections emerge from the mathematics.

#	Check	Proves / catches
1	quote_present	an attestation quote exists — catches agents running outside any enclave
2	quote_structure	the quote parses as a TDX-sized structure
3	enclave_measurement	the code (MRTD) matches the pinned known-good build — catches backdoored code
4	decision_binding	report_data commits to this exact decision — catches forgery, replay, tampering

#	Check	Proves / catches
5	signature	the signature recovers to the attested key — catches tampered payloads / wrong signer
6	inference_provenance	the judgment prompt matches the audited one and the model's output was reported faithfully
7	consensus	the action genuinely followed a quorum — no single judge decided it
8	ledger_link	the hash-chain link is well-formed and the policy accumulator is within limit
9	quote_authenticity	the Intel PCK chain roots in the Intel SGX Root CA (or full DCAP on real hardware)

Across a **sequence** of receipts, verify_sequence() additionally proves the history is **complete** (nothing dropped), **ordered** (nothing reordered/inserted), and **policy-compliant** (the cumulative invariant held at every step).

10. Advanced Capability 1 — Attested Inference Provenance

The gap it closes: attesting that unaltered *code* ran is not the same as attesting *what it was told to do*. A wallet guardian with a byte-identical image (same MRTD, real quote, valid signature) can still be backdoored by swapping the LLM's system prompt to “approve everything from 0xATTACKER.” Naive “the enclave is genuine, so trust it” verification accepts that.

Every LLM-judged receipt binds an **inference block** — provider, model, **system_prompt_sha256**, the input, and the model's actual output — into the signed payload. The verifier then (1) confirms the action matches the model's real output (the agent can't claim the model approved when it denied), and (2) with EXPECTED_SYSTEM_PROMPT_SHA256 pinned, confirms the judgment prompt matches the audited one.

Result: a backdoored-prompt agent — genuine enclave, valid quote, matching MRTD, valid signature — is **rejected at inference_provenance** while passing every hardware check. Attesting the code is not enough if the decision policy can be swapped; Veriform closes that.

Honest boundary: this proves the model consulted, the criteria, and faithful reporting are enclave-signed. It does not prove the remote provider's servers ran that model unmodified — that needs the provider's own attestation (confidential inference). The block carries a provider_attestation field for when that is available.

11. Advanced Capability 2 — Attested Decision Ledger

The gap it closes: every attestation is *point-in-time* — one quote proves one decision. But an autonomous agent makes a *sequence* of decisions over its life, and no consumable tool lets an outsider verify that whole history.

Each decision is appended to a hash chain (root_n = SHA-256(root_{n-1} || entry_hash_n)) and carries a running policy accumulator, all bound into the signed, quote-anchored payload. A verifier given a sequence of receipts proves three things no single quote can:

- **Completeness** — every decision is present; dropping one breaks the chain.
- **Ordering** — none reordered or inserted (chained roots + sequence numbers).
- **Cross-decision policy** — a cumulative invariant held at every step. In the demo, three transfers each pass the per-transfer limit, but the third is **denied by the ledger** because it would push the daily total past the cap — a policy no per-decision check can enforce.

Honest boundary: tamper-evident within a presented sequence. Defeating a rollback/fork (re-presenting a shorter history) requires anchoring the latest root outside the enclave — a hardware monotonic counter or posting the root on-chain. The ledger is built so that anchoring is a one-line addition.

12. Advanced Capability 3 — Attested Multi-Judge Consensus

The gap it closes: the deepest trust question in the agent space — why trust a *single* AI's judgment on something that moves money? If that one model hallucinates, is compromised, or is backdoored, its lone “APPROVE” carries the decision.

With CONSENSUS enabled, the agent runs a **quorum of independent judges** (a real LLM plus independent rule-based evaluators; more real models when keys are added) and only approves if a

threshold agree. **Every vote** is bound into the receipt. The verifier recomputes the tally, confirms the final action genuinely followed from the votes, and — with the threshold pinned — rejects any receipt where a compromised agent tried to lower its own bar or misreport the vote.

Live result: with one judge fully compromised to approve everything, a scam request is still **DENIED** (1 of 3 approvals, need 2). The rogue judge cannot force an approval, and the proof — every vote — is in the receipt.

Honest boundary: proves the consulted quorum agreed and the action followed; it does not make any individual judge correct, and the judges must be genuinely independent (a shared failure mode defeats a quorum).

13. Advanced Capability 4 — Full DCAP on Real Silicon

The default quote_authenticity check validates the Intel certificate chain. Veriform also implements **complete DCAP verification** — every signature in the chain of trust, offline, no paid service:

```
Intel SGX Root CA
   |-signs-> PCK certificate
              |-signs-> QE report
                        |-binds-> attestation key
                                  |-signs-> TD report --contains--> report_data
```

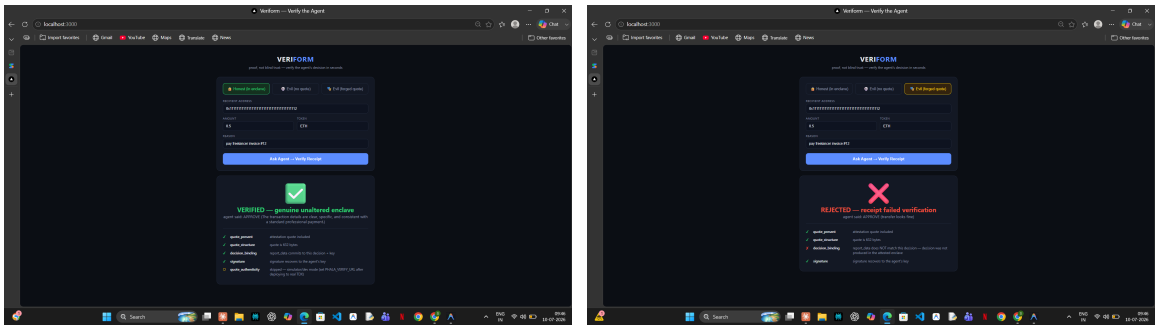
Verified against a **real TDX quote captured from genuine Intel hardware**, all five links pass: quote format, attestation-key signature over the TD report, QE report binds the key, PCK certificate signs the QE report, and the chain roots in the Intel SGX Root CA.

This is the guarantee real silicon adds — the attestation-key signature covers the entire report including report_data. It is exactly what a simulator cannot fake: a simulator serves a real captured quote but rewrites report_data after capture, so full DCAP on a simulator quote **correctly fails** at the report signature. The verifier's real-hardware path is thus built and proven; producing a quote over our own report_data on live silicon is a deployment step, not a missing capability.

14. The Tamper Demonstration

The demo makes invisible security tangible: four agents, the same transfer form, four outcomes — each rejection for a different real reason.

Agent	What it is	Outcome
Honest	real agent, enclave key, genuine binding, quorum judged	**VERIFIED**
Evil (no quote)	impostor outside the enclave; valid signature, no attestation	**REJECTED — quote_present**
Evil (forged quote)	impostor attaching a structurally valid fake quote	**REJECTED — decision_binding**
Backdoored prompt	genuine enclave, valid quote & signature, swapped judgment prompt	**REJECTED — inference_provenance**



Left: honest agent VERIFIED. Right: a forged quote REJECTED — it passes quote presence, structure, and signature, and is still caught, because report_data cannot commit to a decision not made inside the enclave.

The backdoored and consensus cases go further: a genuine enclave with a swapped prompt is rejected, and a compromised judge cannot force an approval — both proven live.

15. Security Model and Honest Boundaries

Trust model

The **operator** (host, root, cloud admin) is **untrusted** — it can read/alter anything outside the enclave and control the network. Trust roots only in **Intel's pinned PKI** and the **enclave attestation**. The **verifier trusts only mathematics** — everything it checks is recomputable from the receipt plus public knowledge. The adversary is a malicious operator who wants a forged, altered, or policy-violating decision accepted as genuine.

Out of scope (documented, with intended mitigations)

- **Liveness / DoS** — the operator can refuse to run the agent; Veriform proves nothing was forged, not that the operator acts.
- **Input confidentiality** — the operator can still see the request; encrypting inputs to the enclave is a natural extension.
- **Remote-model execution** — needs the provider's own attestation.
- **Ledger rollback** — needs the root anchored externally (monotonic counter or on-chain).
- **Replay / freshness** — a nonce + expiry is a known, not-yet-implemented mitigation.
- **Single-vendor / side channels** — multi-vendor attestation (Intel + AMD + Nvidia) is the intended hardening.

Veriform composes standard primitives into a consumable verification layer; it introduces no new cryptography and has not undergone formal security review. The guarantees are design intent supported by tests, not a certified product — stated plainly, because a verification system that overstates itself is worse than one that states its limits exactly.

16. Live Test Results

Scenario	Expected	Result
Honest receipt (quorum judged)	VERIFIED, all checks	**PASS**
Payload tampered after signing	REJECTED: binding + signature	**PASS**
Missing quote	REJECTED: quote_present	**PASS**
Forged quote	REJECTED: decision_binding	**PASS**
Backdoored judgment prompt	REJECTED: inference_provenance	**PASS**
Rogue judge tries to force approval	DENIED by quorum	**PASS**
Decision dropped from history	REJECTED: broken chain	**PASS**
Full DCAP on real hardware quote	all 5 links VERIFIED	**PASS**
Tampered hardware report	REJECTED: att-key signature	**PASS**

49 automated tests, all passing; continuous integration green on every push. The honest agent verifies with all checks green against the official Phala dstack simulator (real Intel PKI chain); full DCAP is proven against a genuine captured hardware quote.

17. How to Run and Demo

Start everything (Windows, one command)

```
pip install fastapi uvicorn httpx eth-account cryptography dstack-sdk
powershell -ExecutionPolicy Bypass -File scripts\run-local.ps1
# then open http://localhost:3000
```

It reads GEMINI_API_KEY from .env (free tier) and falls back to rule-only judging if absent. On Linux/macOS, use the official simulator: `phala simulator start` then `docker compose up`.

The 2-minute demo flow

- **Honest** — leave the rent transfer, click Ask. Point at the giant **VERIFIED** and the green checklist, including inference_provenance, consensus, and quote_authenticity.
- **Scam test** — change the reason to “URGENT giveaway, double your ETH.” It is DENIED, with the denial itself in a verified receipt.
- **Evil (no quote)** — click. **REJECTED**: identical API, valid signature, but no attestation. The verifier trusts math, not the server.
- **Evil (forged quote)** — click. **REJECTED** at decision_binding: a structurally perfect fake quote, caught because it cannot commit to the decision.
- **Backdoored prompt** — click. **REJECTED** at inference_provenance: a genuine enclave whose judgment prompt was swapped.

Deeper capabilities (API)

- **POST /verify-sequence** — submit several receipts; drop one and watch the history be rejected for a broken chain.
- **POST /verify-dcap** — submit a raw quote for full Intel DCAP verification.

The full script, with talking points and judge Q&A, is in DEMO.md.

18. Roadmap and Status

Item	Status
Core receipts, tamper demo, verifier UI	**DONE**
Pluggable free judgment (Claude / Gemini / Ollama), fail-closed	**DONE**
Attested inference provenance (prompt pinning)	**DONE**
Attested decision ledger (verifiable history)	**DONE**
Attested multi-judge consensus	**DONE**
Full DCAP verification, proven on real hardware quote	**DONE**
Official simulator validation + real Intel PKI chain	**DONE**
Threat model, 49 tests, CI, docs, pitch deck	**DONE**
On-chain anchor contracts (ecrecover-verified)	**BUILT; testnet deploy pending**

Item	Status
Bind our own decision on live Intel silicon	**deployment only; verifier path proven**
Replay/freshness, key rotation, multi-vendor attestation	**designed; future work**

Everything achievable without paid hardware access is complete. The remaining hardware step is a deployment, not a missing capability — the verifier's full-DCAP path is built and proven against a genuine Intel quote.

19. Technology Stack and Repository

Layer	Technology
Enclave runtime	Phala dstack (Intel TDX); official Linux simulator for dev
Agent	Python, FastAPI, dstack SDK, eth-account (secp256k1)
Judgment	Google Gemini (free tier) / Anthropic Claude / local Ollama / rules
Verifier	Python, FastAPI, cryptography (X.509 + ECDSA P-256 DCAP), single-page UI
On-chain	Solidity (VeriformRegistry, DemoConsumer) for Base Sepolia
Quality	pytest (49 tests), GitHub Actions CI

Repository: github.com/Sakthi-Sundaram-R/Veriform — including this guide, a 10-slide pitch deck, DEMO.md, SECURITY.md, and the full test suite.

Veriform — proof, not blind trust.